

Prompt Optimization, Evaluation and DSPy

Topics

- e! Introduction to DSPy
- e! DSPy Modules and Signatures
- e! ChainOfThought and Predict Modules
- e! ReAct Modules in DSPy
- e! DSPy Optimizers (BootstrapFewShot, MIPRO)
- e! Compiling and Validating DSPy Programs
- e! Metric Functions and Evaluation
- e! DSPy with Multiple LLMs
- e! Building Self-Improving Pipelines
- e! DSPy vs Manual Prompting Trade-offs
- e! DSPy Production Deployment

Learning Objectives

By the end of this module, you will be able to:

- e! Explain DSPy concepts, workflows, modules, and signatures.
- e! Use Predict, ChainOfThought, and ReAct modules for different AI tasks.
- e! Optimise DSPy programs using BootstrapFewShot and MIPRO.
- e! Compile, validate, and evaluate DSPy programs using task-specific metrics.
- e! Build and deploy self-improving, multi-model DSPy pipelines.

Introduction to DSPy

Introduction to DSPy

DSPy is a framework for building AI workflows using small reusable parts instead of writing one long prompt.

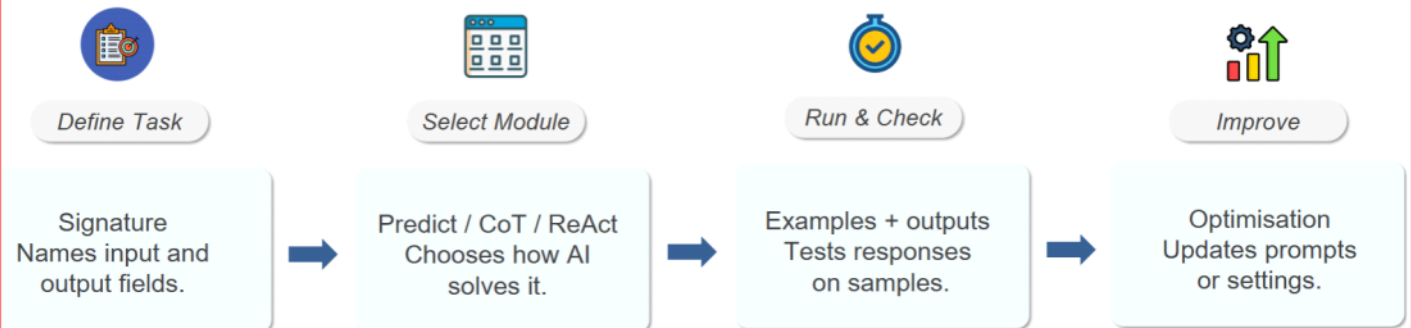


Example

Task: "Summarise a customer review."

DSPy organises the input, model call, and answer in a repeatable way.

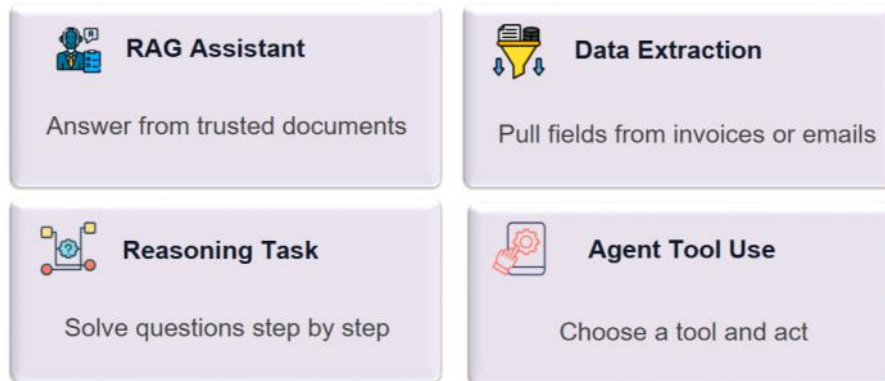
DSPy Workflow at a Glance



DSPy is useful when an AI task should be tested, reused, and improved over time.

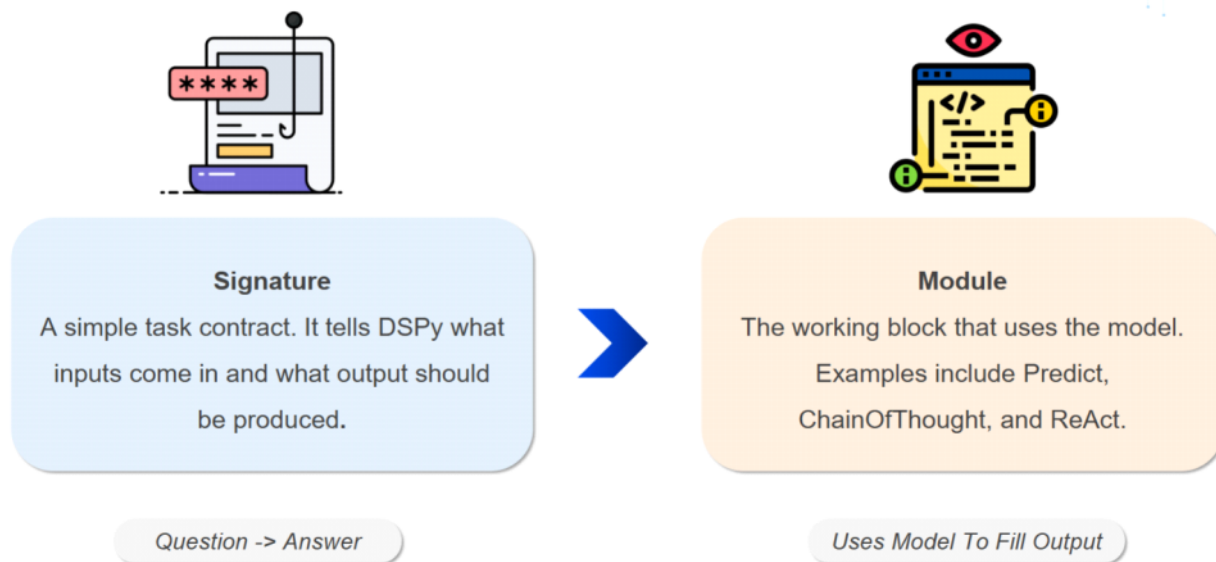
DSPy in Real Projects

DSPy is practical when one AI workflow must run many times with consistent quality.



DSPy Modules and Signatures

DSPy Modules and Signatures



Creating a Simple Signature

```
import dspy

class SummariseReview(dspy.Signature):
    review: str = dspy.InputField()
    summary: str = dspy.OutputField()
```

Code Walkthrough

Input field: customer review text

Output field: short summary

DSPy uses this contract to guide the model.

Good signatures make AI workflows clearer for teams, testers, and future updates.

How Modules Use Signatures

The same signature can be used with different module styles.



Predict: Direct answer generation



ChainOfThought: Reasoning before answer



ReAct: Reason + tool action loop



ChainOfThought and Predict Modules

Predict Module

Predict is the simplest DSPy module for direct input-to-output tasks.

```
summariser = dspy.Predict(SummariseReview)

result = summariser(
    review="Delivery was fast but packaging was damaged."
)

print(result.summary)
```



Predict asks the model to produce the output described by the signature.



Best when the task is simple and does not need step-by-step reasoning.

ChainOfThought Module

ChainOfThought helps the model reason before producing the final answer.
Useful when the model must compare, infer, or explain a decision before answering.



Question



Reasoning



Answer

```
class QA(dspy.Signature):
    question: str = dspy.InputField()
    answer: str = dspy.OutputField()

qa = dspy.ChainOfThought(QA)

response = qa(question="Why did sales drop?")
```

Signature: Defines the question and answer fields.

ChainOfThought module: Adds step-by-step reasoning before generating the answer.

Execution: Processes the question and returns the final response.

Predict vs. ChainOfThought

Predict



Best for: simple and direct tasks

Example: summarise a review

Use when: fast, short answers are enough

ChainOfThought



Best for: step-by-step reasoning

Example: compare two options

Use when: analysis and explanation are needed

Choose Predict for straightforward tasks and ChainOfThought when reasoning or explanation is required.

Predict vs. ChainOfThought

Predict



Best for: simple and direct tasks

Example: summarise a review

Use when: fast, short answers are enough

ChainOfThought



Best for: step-by-step reasoning

Example: compare two options

Use when: analysis and explanation are needed

Choose Predict for straightforward tasks and ChainOfThought when reasoning or explanation is required.

Demonstration 1 : Building a DSPy Signature and Module Pipeline

ReAct Modules in DSPy

ReAct Modules in DSPy

ReAct means Reason + Act. The model thinks about the next step, uses a tool if needed, observes the result, and continues until it can answer.



Example

A research assistant may search documents, read a result, then decide whether another search is required.

ReAct with Tools: Code Pattern

ReAct is useful when the answer depends on external information or actions.

```
def search_kb(query: str) -> str:
    return "Relevant policy details..."

class SupportQA(dspy.Signature):
    question: str = dspy.InputField()
    answer: str = dspy.OutputField()

agent = dspy.ReAct(SupportQA, tools=[search_kb])
result = agent(question="Can I return this item?")
```

What happens

1. Read the question
2. Decide whether a tool is needed
3. Use the tool
4. Read the result
5. Produce the final answer

DSPy Optimizers (BootstrapFewShot, MIPRO)

DSPy Optimizers: Basic Idea

Optimizers improve a DSPy program by tuning its instructions and examples against a metric.



Program

The DSPy pipeline to improve
Example: question → answer



Metric

A scoring rule
Example: answer matches
expected output



Trainset

A few practice examples
Can be small at the beginning

Compiled Program = Better prompts + selected examples

BootstrapFewShot: Learning from Examples

BootstrapFewShot builds useful few-shot examples, keeps the good ones, and places them into the prompt.



Trainset

Small examples



Teacher Model

Generate answers



Metric Check

Keep passing traces



Selected Demos

Add demos to prompt



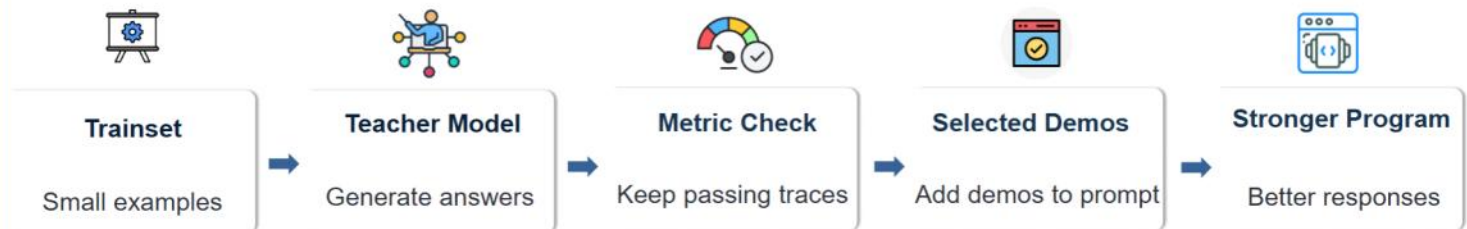
Stronger Program

Better responses

```
optimizer = dspy.BootstrapFewShot(metric=score_answer)
compiled = optimizer.compile(program, trainset=trainset)
answer = compiled(question="What is RAG?")
```

BootstrapFewShot: Learning from Examples

BootstrapFewShot builds useful few-shot examples, keeps the good ones, and places them into the prompt.



```
optimizer = dspy.BootstrapFewShot(metric=score_answer)
compiled = optimizer.compile(program, trainset=trainset)
answer = compiled(question="What is RAG?")
```

MIPRO: Optimising Instructions + Examples

MIPRO explores different instructions and examples, then selects the combination that scores best.

```
optimizer = dspy.MIPROv2(
    metric=score_answer,
    auto="light",
)

compiled = optimizer.compile(
    program,
    trainset=trainset,
)
```

Use when prompt quality needs
systematic improvement

What MIPRO tunes

- Natural-language instructions
- Few-shot examples
- Prompt choices for each module

Key Idea

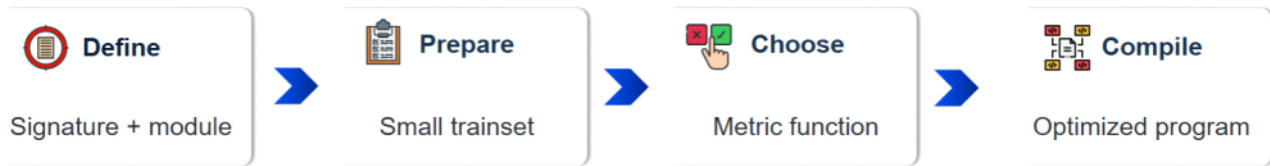
It experiments with different ways of asking the
model, then keeps the version that performs best.

Demonstration 2 :Optimizing Prompts with BootstrapFewShot

Compiling and Validating DSPy Programs

Compiling a DSPy Program

Compiling means creating an improved version of the program using examples and a scoring metric.



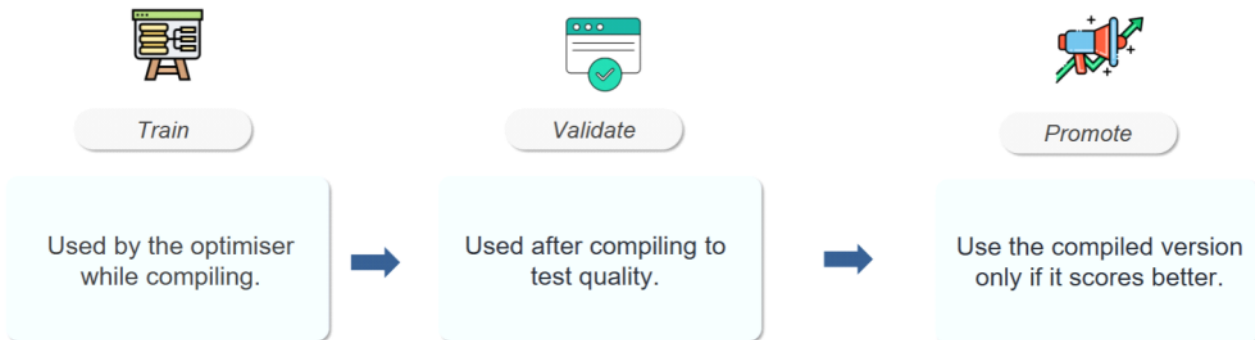
```
class QA(dspy.Signature):
    question = dspy.InputField()
    answer = dspy.OutputField()

    program = dspy.Predict(QA)

compiled = optimizer.compile(program, trainset=trainset)
```

Validating Compiled Programs

Validation checks whether the compiled program works well on examples it did not train on.



Compile + Validate: Small Code Pattern

A simple pattern is: compile with training examples, then evaluate on a separate validation set.

```
from dspy.evaluate import Evaluate

optimizer = dspy.BootstrapFewShot(metric=metric)

compiled = optimizer.compile(
    program,
    trainset=trainset
)

evaluate = Evaluate(
    devset=valset,
    metric=metric,
    display_progress=True
)

score = evaluate(compiled)

print("Validation score:", score)
```

Practical checks

- Score improves on validation examples
- Answers remain grounded and complete
- Output format stays consistent
- No new failures on edge cases

Metric Functions and Evaluation

Metric Functions: Scoring the Output

A metric function tells DSPy what “good” means for the task.

Correctness



Does the answer match the expected result?

Format



Is the response in JSON, bullets, or required structure?

Safety



Does the response avoid unsupported or risky claims?

Writing a Simple Metric Function

Metrics can be simple rule checks or richer scoring functions depending on the task.

Code Walkthrough

```
def answer_exact_match(example, pred, trace=None):
    expected = example.answer.strip().lower()
    actual = pred.answer.strip().lower()
    return expected == actual

def contains_keyword(example, pred, trace=None):
    return example.keyword.lower() in pred.answer.lower()
```

- `answer_exact_match()` : Compares expected and predicted answers
- Text normalization: Removes case and spaces
- Boolean result: Returns True or False
- `contains_keyword()` : Checks required keyword presence
- Custom evaluation metrics: Creates task-specific scoring

Writing a Simple Metric Function

The metric should match the business goal of the application.

Task	Good Metric	Example Check
FAQ Answering	Accuracy / relevancy	Answer includes the correct policy
JSON Output	Format validity	Response parses as JSON
Summarisation	Coverage + length	Includes key facts under word limit
Retrieval QA	Groundedness	Answer uses retrieved context

Demonstration 3 : Writing Metric Functions to Evaluate DSPy Programs

DSPy with Multiple LLMs

DSPy with Multiple LLMs

A DSPy program can use different models for different jobs, such as cheap drafting and stronger judging.



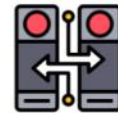
Small Model

Drafting, extraction,
simple classification



Strong Model

Complex reasoning,
grading, final answer



Context Switch

Choose the right
model for the right step

Routing Work Across Models

Use a faster model for routine work and a stronger model where quality matters most.

```
fast_lm = dsp.LM("openai/gpt-4o-mini")
strong_lm = dsp.LM("openai/gpt-4.1")

with dsp.context(lm=fast_lm):
    draft = extractor(text=document)

with dsp.context(lm=strong_lm):
    final = reasoner(
        question=question,
        notes=draft.notes
    )
```

Why this helps

- Controls cost by not using a large model everywhere
- Improves quality for difficult reasoning steps
- Makes pipelines easier to test step by step

Comparing Model Choices

Run the same validation set across model combinations to choose the best cost-quality balance.

Setup	Quality	Cost	Best Use
Small only	Medium	Low	Drafts, tags, extraction
Strong only	High	High	Critical answers
Mixed pipeline	High	Medium	Most production workflows

```
for lm in [fast_lm, strong_lm]:  
    with dspy.context(lm=lm):  
        print(evaluate(program))
```

Building Self-Improving Pipelines

Building Self-Improving Pipelines

A self-improving pipeline repeatedly evaluates results, learns from errors, and recompiles better versions.

Run



Produce outputs

Score



Measure quality

Review



Find weak cases

Improve



Compile again

Release



Use better version

The key is controlled improvement, not random prompt editing.

Self-Improving Pipeline: Code Pattern

Keep a validation score, compare versions, and only promote the new version when it improves.

```
baseline_score = evaluate(program)

compiled = optimizer.compile(
    program,
    trainset=trainset
)

new_score = evaluate(compiled)

if new_score > baseline_score:
    compiled.save("qa_program_v2.json")
    active_program = compiled
else:
    active_program = program
```

Safe improvement rules

- Never replace a working version without validation
- Keep saved versions for rollback
- Track scores across test cases
- Review failures before increasing traffic

Example: Customer Support QA Pipeline

A support QA pipeline can improve over time using new tickets, scores, and failed examples.

Before optimisation

- Answers policy questions inconsistently
- Misses required refund conditions
- Uses too much wording in simple

After DSPy optimisation

- Uses better few-shot examples
- Scores answers with a policy metric
- Promotes the version with higher validation score

```
metric = policy_score

optimizer = dspy.MIPROv2(metric=metric, auto="light")

compiled_support_bot = optimizer.compile(bot, trainset=ticket)
```

DSPy vs. Manual Prompting Trade-offs

DSPy vs. Manual Prompting

A practical comparison between hand-written prompts and metric-driven optimisation.



Manual prompting

You write, test, and edit prompt text by hand until the output looks acceptable.



DSPy DSPy approach

You define the task, examples, and metric. DSPy compiles the program to improve prompts or examples.

Write prompt



Try output



Score with metric



Improve

Manual prompting is fast for a first draft. DSPy is better when quality needs to be measured and improved repeatedly.

Trade-offs: Speed, Control, and Quality

Choose the approach based on project stage, data availability, and evaluation needs.

Manual Prompting Works Best When

Early exploration

You are still discovering the task.

Small prototype

Only a few examples are needed.

Human judgement

A person can quickly check quality.

DSPy Works Best When

Repeated task

The same workflow runs many times.

Clear metric

You can score good vs poor output.

Training examples

You have sample inputs and answers.

DSPy Production Deployment

DSPy Production Deployment

Production means moving from notebook experiments to a stable, monitored application.

1. Compile offline

Optimise the DSPy program using a controlled training set and validation set.

2. Save the program

Store the compiled program version so the same behaviour can be reused.

3. Serve through API

Load the saved program inside an application endpoint or service.

Deployment View



Training examples



DSPy compile



Saved program



API service



Users

Production Code Pattern

A simple pattern: optimise once, save, then load for serving.

```
Training Setup

import dspy

# 1. Define the task
class SupportQA(dspy.Signature):
    question = dspy.InputField()
    answer = dspy.OutputField()

program = dspy.ChainOfThought(SupportQA)

# 2. Optimize with examples and a metric
optimizer = dspy.BootstrapFewShot(metric=my_metric)
compiled = optimizer.compile(
    program,
    trainset=trainset
)

# 3. Save for production
compiled.save("support_qa_v1.json")
```

```
Serving Step

import dspy

# 4. Load the saved program
app = dspy.load("support_qa_v1.json")

# 5. Use it in an API route
def answer_user(question: str):
    result = app(question=question)
    return {
        "answer": result.answer,
        "version": "support_qa_v1"
    }
```

Operational Practices for DSPy



Versioning

Track every compiled program

Save program name, version, training data snapshot, metric score, and release date.

Monitoring

Watch live behaviour

Log latency, cost, failures, user feedback, and low-confidence outputs.

Fallbacks

Avoid silent failure

Use a safe baseline prompt, human review, or simpler model path when needed.

Signal	Healthy example	Risk example	Action
Quality	Metric stable	Score drops	Rollback / retrain
Latency	Fast response	Slow output	Route / scale
Cost	Within budget	Token spike	Limit / alert

Summary

In this lesson, you have learned to:

- ! DSPy workflows, modules, signatures, and reusable AI programs.
- ! Predict, ChainOfThought, and ReAct modules for generation and tool use.
- ! BootstrapFewShot and MIPRO for optimising prompts and examples.
- ! Compiling, validating, and evaluating programs with metric functions.
- ! Multi-model pipelines and controlled self-improvement workflows.
- ! DSPy trade-offs, deployment patterns, monitoring, versioning, and fallbacks.